



Python Fundamentals II

Software Development COS7009–B

Contents

While loop (Round II)

Variable – Object

Type Conversion

Relational Operators

While Loop, Round Two

- while loop, oddly, can have an associated `else` suite
- `else` suite is executed when the loop finishes under normal conditions
 - Basically the last thing the loop does as it exits



while With else (1 of 2)

```
while booleanExpression:
```

```
    suite
```

```
    suite
```

```
else:
```

```
    suite
```

```
    suite
```

```
rest of the program
```

while With else (2 of 2)

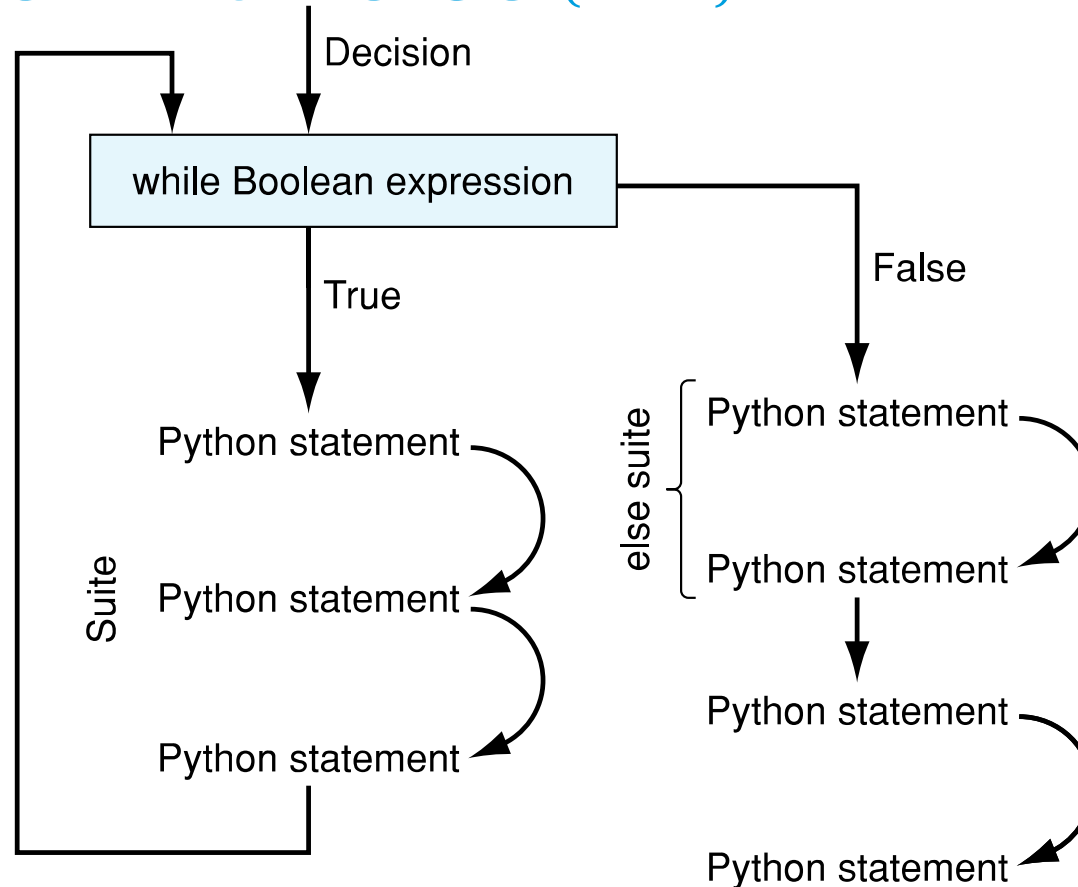


FIGURE 2.9 *while-else*.

Break Statement

- A `break` statement in a loop, if executed, exits the loop
- It exists immediately, skipping whatever remains of the loop as well as the else statement (if it exists) of the loop



Continue Statement

- A `continue` statement, if executed in a loop, means to immediately jump back to the top of the loop and re-evaluate the conditional
- Any remaining parts of the loop are skipped for the one iteration when the `continue` was executed



Change in Control : Break and Continue

- While loops are easiest read when the conditions of exit are clear
- Excessive use of continue and break within a loop suite make it more difficult to decide when the loop will exit and what parts of the suite will be executed each loop.
- Use them judiciously.

While Overview

while test1:

 statement_list_1

 if test2: break

 # Exit loop now; skip else

 if test3: continue

 # Go to top of loop now

 # more statements

else:

 statement_list_2

 # If we didn't hit a 'break'

'break' or 'continue' lines can appear anywhere

Contents

While loop (Round II)

Variable – Object

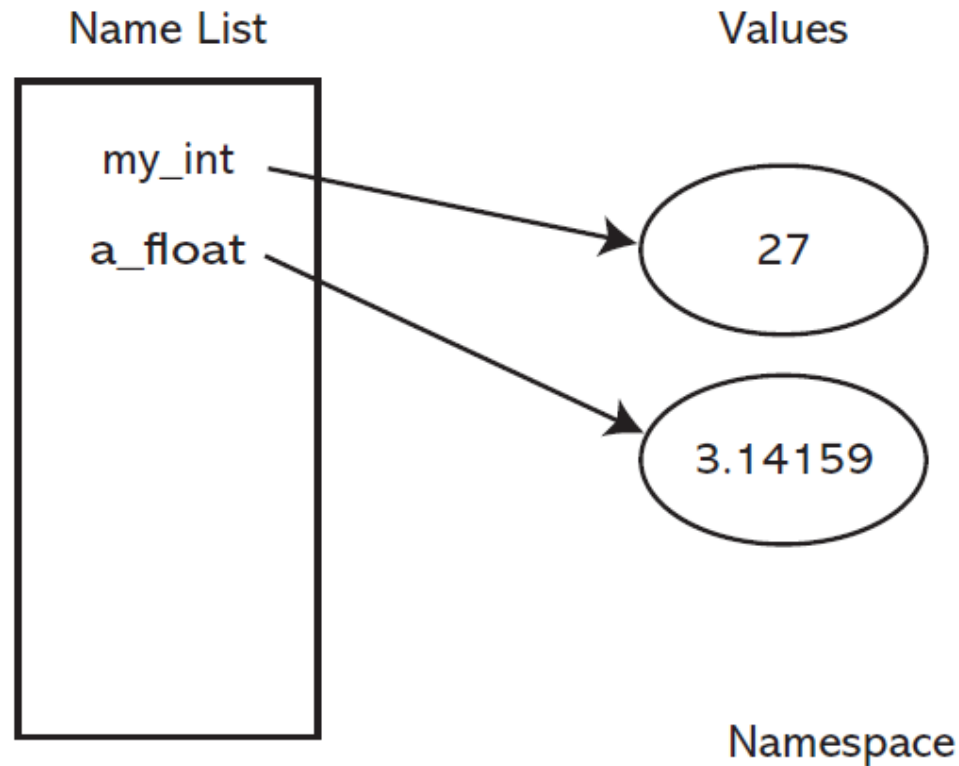
Type Conversion

Relational Operators

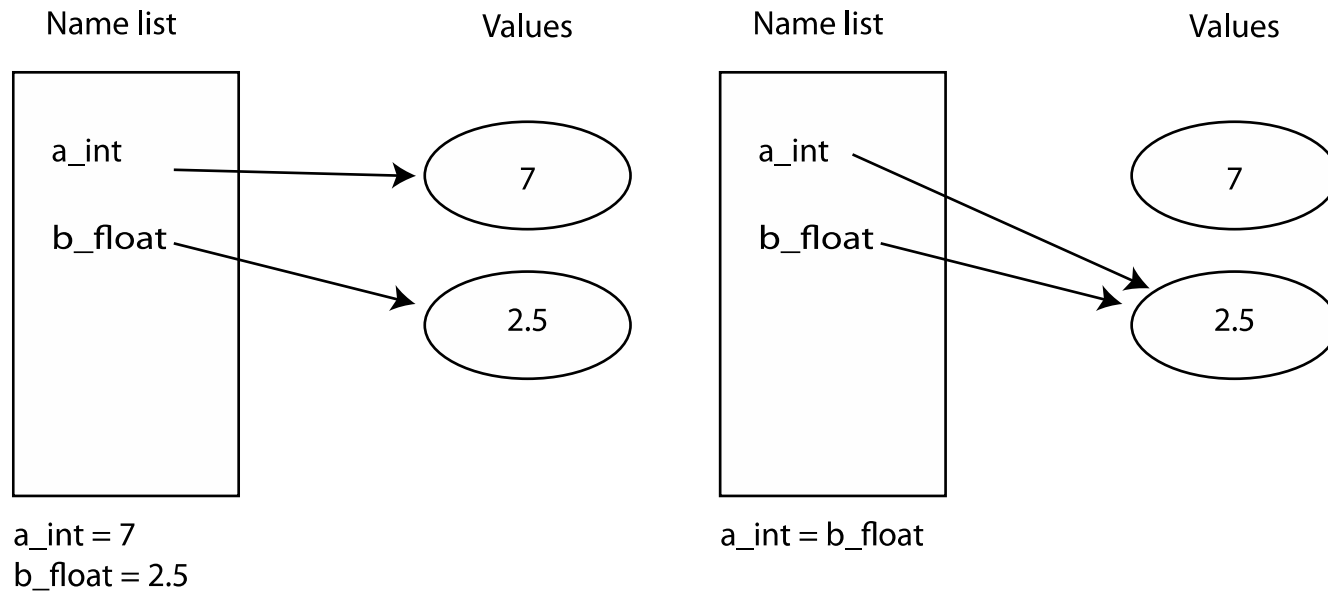
Variable name

- Naming convention
 - Every name must begin with a letter or the underscore character
 - A numeral is not allowed as the first character
 - Case sensitive

Variable Creation and Assignment



Objects and Types



Can do Multiple Assignments

```
a_int, b_int = 2, 3
```

first on right assigned to first on left, second on right
assigned to second on left

```
print(a_int, b_int)    # prints 2 3
```

```
a_int, b_int = 1, 2, 3 → Error
```

counts on lhs and rhs must match

Traditional Swap

- Initial values: `a_int = 2, b_int = 3`
- Behavior: swap values of `x` and `y`
 - Note: `a_int = b_int`
`b_int = a_int` doesn't work (why?)
 - introduce extra variable `temp`
 - `temp = a_int` # save `a_int` value in `temp`
 - `a_int = b_int` # assign `a_int` value to `b_int`
 - `b_int = temp` # assign `temp` value to `b_int`

Swap Using Multiple Assignment

```
a_int, b_int = 2, 3
```

```
print(a_int, b_int) # prints 2 3
```

```
a_int, b_int = b_int, a_int
```

```
print(a_int, b_int) # prints 3 2
```

remember, evaluate all the values on the rhs first, then assign to variables on the lhs

Chaining for Assignment

Unlike other operations which chain left to right, assignment chains right to left

```
a_int = b_int = 5
```

```
print(a_int, b_int) # prints 5 5
```

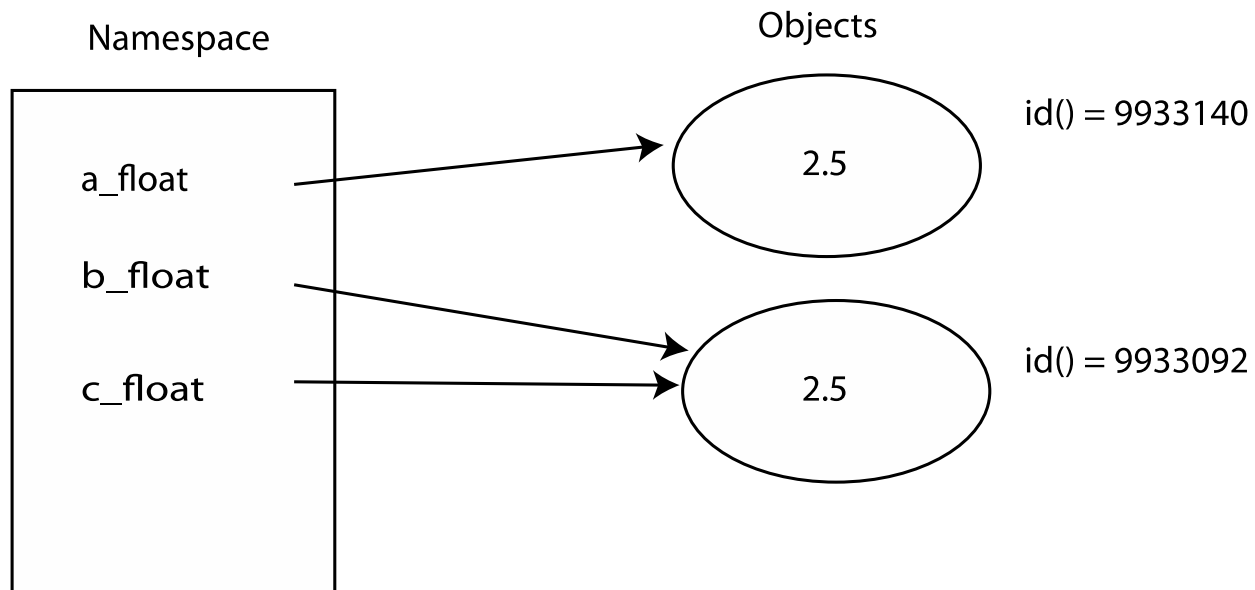
What Does Equality Mean? (1 of 2)

Two senses of equality

- Two variables refer to different objects, each object representing the same value
- Two variables refer to the same object. The `id()` function used for this.

What Does Equality Mean? (2 of 2)

```
a_float = 2.5  
b_float = 2.5  
c_float = b_float
```



Equal vs. Same

- `==` compares values of two variable's objects, do they represent the same value
- `is` operator determines if two variables are associated with the same value

`a_float == b_float -> ??`

`a_float is b_float -> ??`

`b_float is c_float -> ??`

Contents

While loop (Round II)

Variable – Object

Type Conversion

Relational Operators

Python “types”

- Integers: 5
- Floats: 1.2
- Strings: “anything” or ‘something’
- Lists: [,] ['a',1,1.3]
- More types are available as third-party add-ons

Integer and Floating

- Leading zero without letter is invalid
 - E.g. 012
- Leading zero with a letter is to encode a number in a base other than 10
 - E.g. 0x base 16, 0b base 2
- Floating-point values yield approximate values
 - E.g. $1.1 + 2.2$

Converting Types

- A character '1' is not an integer 1.
- You need to convert the value returned by the `input` command (characters) into an integer
- `int("123")` yields the integer 123

Type Conversion

- `int(some_var)` returns an integer
- `float(some_var)` returns a float
- `str(some_var)` returns a string
- should check out what works:
 - `int(2.1) → 2`, `int('2') → 2`, but `int('2.1')` fails
 - `float(2) → 2.0`, `float('2.0') → 2.0`, `float('2') → 2.0`,
`float(2.0) → 2.0`
 - `str(2) → '2'`, `str(2.0) → '2.0'`, `str('a') → 'a'`

Contents

While loop (Round II)

Variable – Object

Type Conversion

Relational Operators



Relational Operators

- $3 > 2 \rightarrow \text{True}$
- Relational Operators have low preference
 - $5 + 3 < 3 - 2$
 - $8 < 1 \rightarrow \text{False}$
- $'1' < 2 \rightarrow \text{Error}$
 - Can only compare like types
- $\text{int}('1') < 2 \rightarrow \text{True}$
 - Like types, regular compare

Chained Comparisons

- In Python, chained comparisons work just like you would expect in a mathematical expression:
- Given myInt has the value 5

`0 <= myInt <= 5 -> ??`

`0 < myInt <= 5 > 1 -> ??`

`4 < 5 == True -> ??`

Pitfall

Floating point arithmetic is approximate! Try:

$$1.1 + 2.2 == 3.3 \rightarrow ??$$

Also:

```
>>> u = 111111113
>>> v = -111111111
>>> w = 7.511111111
>>> x = (u + v) + w
>>> y = u + (v + w)
?? >>> x == y
```

Compare Using “Close Enough”

Establish a level of “close enough” for equality

```
>>> u = 111111113
>>> v = -111111111
>>> w = 7.511111111

>>> x = (u + v) + w
>>> y = u + (v + w)
>>> x == y
```

False

```
>>> abs(x - y) < 0.0000001  # abs is absolute value
```

True



Boolean Operators versus Relationals

- Relational operations always return `True` or `False`
- Boolean operators (`and`, `or`) are different in that:
 - They can return values (that represent `True` or `False`)
 - They have **short circuiting**
- `0`, `' '`, `[ ]` or other “empty” objects are equivalent to `False`
- anything else is equivalent to `True`